Ders 4 Genişletilmiş Notu: Uygulama: Pisagor Üçlüsü ve 2B Konvolüsyon; Örüntü Eşleme, Matriste Arama ve Çok Boyutlu Diziler

Ders 4 Genişletilmiş Notu: Uygulama: Pisagor Üçlüsü ve 2B Konvolüsyon; Örüntü Eşleme, Matriste Arama ve Çok Boyutlu Diziler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Birinci Uygulama: Pisagor Üçlüsü
- 2. İkinci Uygulama: İki Boyutlu Konvolüsyon
- 3. Konvolüsyon Filtreleri ve Uygulama Alanları
- 4. Örüntü Eşleme (Pattern Matching)
- 5. Matris İçinde Örüntü Arama
- 6. Çok Boyutlu Diziler ve İşaretçiler

Dersin ilk yarısı uygulamadır: Arş. Grv. Muhammet Taha Gökcan, o haftaya kadar görülen konuları içeren iki soru çözdü. İkinci yarıda Dr. Ahmet Elbir konvolüsyonun nerede kullanıldığını gösterdi, ardından string ve matris konularıyla devam etti.

> **Ders sonrası not:** Asistanın sesi kayıtta yer yer kesik, kod yazdığı bölümlerde altyazı çoğu zaman okunamıyor. 1. ve 2. bölümdeki kodlar, kısa nottaki ekran okumasından ve anlatımdan kuruldu, `gcc -Wall` ile derlenip çalıştırıldı. Değişken adları kısa nottaki gibidir; ekrana yazılan mesajlar temsilîdir.

1. Birinci Uygulama: Pisagor Üçlüsü

1.1 Problem

Verilen bir dizide $a^2 + b^2 = c^2$ eşitliğini sağlayan üç sayı var mı, varsa hangileri, bunu bulup ekrana yazan bir program yazılacak. Örnek dizi şudur: 3, 13, 5, 6, 4, 12, 8.

Herhangi bir üçlü bu eşitliği sağlayabileceği için dizideki bütün üçlü kombinasyonlara bakmak gerekir. Asistan önce sınıfa sorunun daha kolay hâlini sordu ve birkaç dakika düşünme süresi verdi.

Soru: Bir dizideki bütün ikili kombinasyonları bulan bir program nasıl yazılır? **Cevap:** Geçen dönem görülen seçmeli sıralamaya (selection sort) benzer bir yapı kurulur. İki indis vardır ve ikinci indis her zaman birincinin sağından başlar.

1.2 İkili Kombinasyonlar

İlk indis `i` birinci elemanda, yani 3'te sabit durur. İkinci indis `j` ondan sonraki elemandan başlar ve dizinin sonuna kadar ilerler. Böylece 3 ile başlayan bütün ikililer bulunur: 3-13, 3-5, 3-6 ve devamı. Sonra `i` bir artırılır ve 13'e geçilir, artık 13'ün geri kalan elemanlarla kombinasyonlarına bakılır.

Burada `j`'yi yeniden baştan başlatmanın anlamı yoktur. 3-13 kombinasyonuna zaten bakılmıştı. `j` baştan başlarsa aynı iş tekrarlanır. Bu yüzden `j` her zaman `i`'den sonraki elemandan başlar. İşlem, `i` ve `j` son iki elemana, yani 12-8 ikilisine ulaştığında biter. Seçmeli sıralamada da en küçük eleman bulunup başa alındıktan sonra yalnız geri kalan elemanlara bakılıyordu, benzerlik buradadır.

> **Hocanın vurgusu:** Bütün kombinasyonları denerken her indisi bir öncekinin bir sonrasından başlatın, aynı kombinasyonu tekrar denemeyin.

1.3 Üçlü Kombinasyonlar

Üçlü kombinasyonda üçüncü bir indis `k` eklenir ve iş biraz daha karışır. Adımlar şöyledir:

1. `i` 3'te, `j` 13'te durur. `k`, `j`'den sonraki elemandan başlar. İlk üçlü 3, 13, 5'tir.
2. `k` artırılır: 3, 13, 6 ve devamı. `k` dizinin sonuna ulaşınca 3-13 ile başlayan bütün üçlüler bitmiştir.
3. `j` bir artırılır, sıra 3-5 ile başlayanlara gelir. `k` yine baştan başlamaz, çünkü aynı kombinasyonu bulmanın anlamı yoktur. `k`, `j`'den bir sonraki elemandan başlar.
4. `i`, `j` ve `k` sona ulaştığında bütün üçlü kombinasyonlar denenmiş olur.

1.4 Üçlünün Sırası

Üçlüler dizideki sırayla geldiği için eşitliği sağlayan bir üçlü, $a^2 + b^2 = c^2$ biçimine uygun sırada gelmeyebilir. Örneğin sırayla ilerlerken 3, 5, 4 gelir; hipotenüs olan 5 ortadadır. Bu yüzden her üçlüde üç ihtimalin hepsi denenir: üçlüdeki her sayı sırayla c yerine konur. Elemanların kareleri $x = a_i^2$, $y = a_j^2$, $z = a_k^2$ olmak üzere koşul şudur:

$$x + y = z \quad \text{veya} \quad x + z = y \quad \text{veya} \quad y + z = x$$

1.5 Kod

Program standart kütüphaneyi ekleyerek başlar. Sayıları tutmak için bir dizi gerekir. Boyut doğrudan sayı olarak yazılabilirdi, ama kod daha genel (generic) olsun diye `#define MAX 10` makrosu kullanıldı. Makro, derlemeden önce kullanıldığı yere yazılır.

Kullanıcıya önce `printf` ile dizinin boyutu sorulur ve kaç eleman gireceği `scanf` ile alınır. Tanımlanan dizinin en büyük boyutu bellidir, kullanıcının gireceği eleman sayısı ise değişebilir. Ardından `i` sıfırdan başlayan bir döngüyle elemanlar tek tek okunur.

Elemanı okurken `scanf`'e `&Array[i]` yazılır. Amaç, o hücrenin adresini `scanf`'e vermektir. Doğrudan `Array[i]` yazılsaydı fonksiyona adres değil, hücredeki değer gitmiş olurdu. `&` ile adres verilince `scanf` değeri nereye yazacağını bilir.

Karelerin her seferinde uzun uzun yazılmaması için `x`, `y`, `z` değişkenleri kullanıldı. Koşul sağlanırsa bulunan üçlü sayacı `s` artırılır ve üçlü yazdırılır. Döngüler bitince üçlü hiç bulunmadıysa bunu söyleyen bir mesaj verilir.

```c
#include <stdio.h>
#define MAX 10

int main(void)
{
    int Array[MAX];
    int N, i, j, k;
    int x, y, z;
    int s = 0;

    printf("Dizinin boyutunu girin: ");
    scanf("%d", &N);

    for (i = 0; i < N; i++) {
        printf("%d. elemani girin: ", i + 1);
```

```
        scanf("%d", &Array[i]);
    }

    for (i = 0; i < N; i++)
        for (j = i + 1; j < N; j++)
            for (k = j + 1; k < N; k++) {
                x = Array[i] * Array[i];
                y = Array[j] * Array[j];
                z = Array[k] * Array[k];

                if (x + y == z || x + z == y || y + z == x) {
                    s++;
                    printf("\nPisagor uclusu: %d %d %d",
                           Array[i], Array[j], Array[k]);
                }
            }

    if (s == 0)
        printf("\nPisagor uclusu bulunamadi");
    printf("\nToplam: %d\n", s);

    return 0;
}
```

Asistan söz dizimiyle ilgili bir hatırlatma da yaptı: `if` veya `for` gövdesinde tek bir satır varsa süslü parantez zorunlu değildir, birden fazla satır varsa gerekir. Yukarıdaki iç içe `for`'lar bu yüzden parantezsiz yazılabilir; en içteki `for` ise birkaç satır içerdiği için parantez ister.

Derste program derlenirken bir satır sonunda `;` unutulduğu için `expected ';' before 'y'` hatası alındı ve düzeltildi (hata metni kısa nottaki ekran okumasından alınmıştır).

> **Ders sonrası not:** Örnek dizi için program `gcc -Wall` ile derlenip çalıştırıldı, iki üçlü buldu: (3, 5, 4) ve (13, 5, 12). Kontrol python ile hesaplandı: $3^2 + 4^2 = 25 = 5^2$ ve $5^2 + 12^2 = 169 = 13^2$.

2. İkinci Uygulama: İki Boyutlu Konvolüsyon

2.1 İşlem Nasıl Yapılır?

Konvolüsyon, görüntü işlemede çok kullanılan bir işlemdir. Elimizde bir girdi (input) matrisi vardır. Bu matristen özellik çıkarmak için bir çekirdek (kernel) matrisi kullanılır. Çekirdek önce girdinin sol üst köşesine oturtulur. Üst üste gelen elemanlar çarpılır, çarpımlar toplanır ve sonuç çıktı (output) matrisine yazılır.

Sonraki adımda çekirdek bir sağa kaydırılır, aynı işlem yapılır, sonuç çıktı matrisine yazılır. Çekirdek satırın sonuna gelip daha ileri gidemeyince, çünkü orada değer yoktur, bir alt satıra geçer. Aynı çarpma, toplama ve kaydırma orada da yapılır. Bir alt satıra da inilemeyince işlem biter.

2.2 Sayısal Örnek

Slayttaki örnekte 4×4 girdi ve 3×3 çekirdek vardır:

$$\text{input} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \qquad \text{kernel} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & -1 & 1 \\ -2 & 1 & 1 \end{bmatrix}$$

Asistan sol üst konumda çarpımları sözlü olarak saymaya başladı: 1 ile 1, 2 ile 0, 3 ile 1, 5 ile 0, 6 ile çekirdeğin orta elemanı, 7 ile 1. Son satırın çarpımları ve diğer üç konum (ara adım eklendi):

- Sol üst: $1 + 0 + 3 + 0 - 6 + 7 - 18 + 10 + 11 = 8$
- Bir sağa kayınca: $2 + 0 + 4 + 0 - 7 + 8 - 20 + 11 + 12 = 10$
- Bir alt satırın başı: $5 + 0 + 7 + 0 - 10 + 11 - 26 + 14 + 15 = 16$
- Bir alt satırda bir sağa kayınca: $6 + 0 + 8 + 0 - 11 + 12 - 28 + 15 + 16 = 18$

$$\text{output} = \begin{bmatrix} 8 & 10 \\ 16 & 18 \end{bmatrix}$$

Ders sonrası not: Kısa nota göre slaytta çıktı 2, 2, 2, 2 olarak gösteriliyor. Python ile hesaplandı, `gcc -Wall` ile derlenen kod da aynı sonucu verdi: çıktı $\begin{bmatrix} 8 & 10 \\ 16 & 18 \end{bmatrix}$'dir. Çekirdek katsayılarının toplamı 2 olduğu için bu girdide bir sağa kayınca sonuç 2, bir alt satıra inince 8 artar; dört eleman eşit olamaz. Kendi kodunuzu slayttaki değerlerle sınamayın.

2.3 Çıktı Boyutu

Soru: Çıkış matrisinin boyutu ne olmalı? **Cevap:** Konvolüsyonu şöyle hayal etmek kolaylık sağlar: Çekirdeğin ortasındaki eleman girdideki bir pikselin üstüne gelir ve sonuç o pikselin yerine yazılır. Yani işlem, ortadaki pikselin çevresiyle birlikte işleme sokulmasıdır.

Bu gözle bakınca kenarlardaki piksellerin sorunu görülür. En üst satırdaki bir pikseli merkeze alsak, üstünde karşılık gelen değer yoktur; aynısı alt satır ve yan sütunlar için de geçerlidir. Bu yüzden 3×3 çekirdekte her kenardan birer satır ve sütun kaybolur. 4×4 girdide yalnız ortadaki dört piksel işleme girebilir, çıktı 2×2 olur.

Çekirdek boyutu 3 iken yukarıdan ve aşağıdan toplam 2 satır gider. Boyut 5 olsaydı her kenardan 2, toplamda 4 satır giderdi; 7 olsaydı toplamda 6 satır. Genel kural şudur:

$$\text{output_size} = \text{input_size} - \text{kernel_size} + 1$$

4×4 girdi ve 3×3 çekirdek için $4 - 3 + 1 = 2$ bulunur.

Çekirdeğin kenar uzunluğu genelde tek sayıdır (3, 5 gibi), çünkü bir merkez piksel ve onun çevresi üzerinden işlem yapılır. Çift boyut da kullanılabilir, ama tercih edilmez.

2.4 Konvolüsyon Ne İşe Yarar?

Burada Dr. Ahmet Elbir söz alıp işlemin arka planını anlattı ve sohbete paylaştığı bir bağlantıdaki sayfayı açtırdı. Konvolüsyonu bir operasyon, bir operatör gibi düşünebiliriz. Size tek ya da iki boyutlu bir sinyal verildiğinde bunun üzerinden ayırt edici özellikler elde etmeniz gerekebilir, bunun için de bu tarz filtrelere ihtiyaç duyulur.

Elimizde bir girdi görüntüsü ve kernel, filtre ya da çekirdek dediğimiz bir fonksiyon vardır. Bu da bir matristir ve genellikle 3×3 veya 5×5 seçilir. Çekirdeği resim üzerinde gezdirerek resimdeki gürültüyü giderebilir veya resimden kenar bilgisi, köşe bilgisi, ortalama renk bilgisi gibi farklı bilgiler elde edebilirsiniz.

Bu bilgiler yapay zekâ ve makine öğrenmesi çalışmalarında tahmin ve sınıflandırma için doğrudan kullanılır. Resmin orijinalini doğrudan kullanmak her zaman doğru fikir vermeyebilir, bu yüzden bir ön işlem olarak filtreleme yapılır. İşlemin tek boyutlu hâli de vardır: bu kez 3×3'lük bir matris değil, 3 ya da 5 elemanlı bir dizi, başka bir dizi üzerinde gezdirilir.

Hoca, işlemi matematikte bir ağırlıklı aritmetik ortalamaya benzetti: çekirdekteki katsayılarla çarpıp toplarsınız ve katsayıların toplamına bölersiniz.

Ders sonrası not: Hoca konvolüsyonu "katsayıların toplamına bölünen ağırlıklı ortalama" diye özetledi; bu yalnız bulanıklaştırma gibi katsayıları pozitif olan filtreler için geçerlidir. 3. bölümdeki köşegen çekirdeğinin katsayı toplamı 0'dır, sıfıra bölünemez. Keskinleştirme çekirdeğinin toplamı 1'dir, bölmenin etkisi yoktur. Bu derste yazılan kodda da bölme yoktur, yalnız çarpımlar toplanır.

Ek bilgi (derste anlatılmadı): Kısa nota göre matematikteki konvolüsyonda çekirdek önce 180° döndürülür; derste ve kodda ise döndürülmeden çarpılıyor, bu işlemin adı büyük olasılıkla çapraz korelasyondur (cross-correlation). Derin öğrenme kütüphanelerinde büyük olasılıkla bu işleme yine de konvolüsyon denir. Python ile hesaplandı: 2.2'deki çekirdek 180° döndürülürse sonuç $\begin{bmatrix} 16 & 18 \\ 24 & 26 \end{bmatrix}$ olur. 3. bölümdeki $1, 2, 4$ katsayılı simetrik çekirdekte iki işlem aynı sonucu verir.

2.5 Girdi Kontrolü: `do-while`

Program önce girdi boyutunu, çekirdek boyutunu alır ve çıktı boyutunu hesaplar. Sonra bu değerlerin doğru girilip girilmediği kontrol edilir. Üç hatalı durum vardır:

- `input_size > MAX`: girdi, tanımlanan en büyük boyutu aşıyor.
- `kernel_size % 2 == 0`: çekirdek boyutu çift, oysa tek olması isteniyor.
- `output_size < 1`: çekirdek girdiden çok büyük girilmiş, çıktı boyutu en az 1 olmalı.

Hatalı girişte değerleri yeniden istemek gerekir.

Soru: Okuma kodunu kopyalayıp bir kez daha yapıştırmak yerine, kullanıcı doğru girene kadar yeniden sormak için hangi yapı kullanılabilir? `while` da yazılabilir, ama o zaman okuma kodu iki kez yazılır. **Cevap:** `do-while`. İşlem bir kez gerçekleşir, sonra koşul değerlendirilir, koşul hata gösteriyorsa baştan başlanır.

> **Hocanın vurgusu:** Kullanıcı girdisini kontrol edin; bunun için okuma kodunu çoğaltmak yerine `do-while` kullanın.

2.6 Hesaplama: Dört İç İçe Döngü

Çekirdek hem girdinin satırlarında hem sütunlarında bir matris gibi gezer ve üst üste geldiği yerde bütün elemanlar çarpılır, bu yüzden çekirdeğin kendi elemanlarının da gezilmesi gerekir. Sonuçta döngü sayısı dörttür:

- `i`, `j`: çıktı matrisinin satırı ve sütunu, yani çekirdeğin girdideki konumu. `output_size` kadar döner.
- `ki`, `kj`: çekirdeğin satırı ve sütunu. `kernel_size` kadar döner.

Her çıktı elemanı için ayrı bir toplam çıkar, bu yüzden `sum` her çıktı konumunda sıfırlanır. Girdide doğru elemanı yakalamak için konum ile çekirdek indisi toplanır: `input[i + ki][j + kj]`. Böylece çekirdekte ilerlenirken girdide de eş zamanlı ilerlenir ve karşılıklı elemanlar çarpılmış olur.

Sonuç çıktı matrisine işaretçi aritmetiğiyle yazıldı: `*(*(output + i) + j) = sum;`. Bu, `output[i][j] = sum` ile aynıdır. Asistan burada işaretçi aritmetiğinin ana fikrini hatırlattı: Bir işaretçiye 1 eklendiğinde adres, değişkenin tipine göre farklı adımlarla ilerler. Tip `int` ise derleyici değişkenin boyutunu bildiği için 4 bayt ilerler. Önce `output + i` ile satır seçilir, ardından `+ j` ile o satırda ilerlenir, sonra yıldızla oradaki hücreye erişilir.

```c
#include <stdio.h>
#define MAX 10

int main(void)
{
    int input[MAX][MAX], kernel[MAX][MAX], output[MAX][MAX];
    int input_size, kernel_size, output_size;
    int i, j, ki, kj, sum;

    do {
        printf("Giris boyutu: ");
        scanf("%d", &input_size);
        printf("Cekirdek boyutu: ");
        scanf("%d", &kernel_size);
        output_size = input_size - kernel_size + 1;
    } while (input_size > MAX || kernel_size % 2 == 0 || output_size < 1);
```

```
    for (i = 0; i < input_size; i++)
        for (j = 0; j < input_size; j++)
            scanf("%d", &input[i][j]);

    for (i = 0; i < kernel_size; i++)
        for (j = 0; j < kernel_size; j++)
            scanf("%d", &kernel[i][j]);

    for (i = 0; i < output_size; i++)
        for (j = 0; j < output_size; j++) {
            sum = 0;
            for (ki = 0; ki < kernel_size; ki++)
                for (kj = 0; kj < kernel_size; kj++)
                    sum += input[i + ki][j + kj] * kernel[ki][kj];
            *(*(output + i) + j) = sum;
        }

    for (i = 0; i < output_size; i++) {
        for (j = 0; j < output_size; j++)
            printf("%4d", output[i][j]);
        printf("\n");
    }

    return 0;
}
```

Ders sonrası not: Kısa nota göre ekranda bir süre `printf("%d", &kernel_size);` satırı görülüyor. Çekirdek boyutunu okumak için `scanf("%d", &kernel_size);` olmalıdır. `gcc -Wall` bu satır için `%d` ile `int *` uyuşmazlığı uyarısı verdi. Yukarıdaki kod `gcc -Wall` ile uyarısız derlendi. Hatalı boyutlarla (çift çekirdek, MAX'tan büyük girdi, girdiden büyük çekirdek) denendiğinde `do-while` değerleri yeniden istedi.

3. Konvolüsyon Filtreleri ve Uygulama Alanları

Aradan sonra Dr. Ahmet Elbir konuya döndü. Görüntü konvolüsyonu iki boyutta da, tek boyutta da olur: bir dizinin üzerinde başka bir diziyi, bir matrisin üzerinde başka bir matrisi, hatta üç boyutlu bir dizi düşünürsek onun içinde üç boyutlu bir şeyi baştan sona gezdirmektir. Bugünkü yapay zekâ ve derin öğrenme bunun üzerinden çok ilerliyor; bu ağlara evrişimli sinir ağları (convolutional neural networks) denir.

3.1 Keskinleştirme

Bir girdi görüntüsünde belli çizgilerin keskinleşmesini istiyorsanız bir keskinleştirme (sharpen) çekirdeği gezdirirsiniz. Bu çekirdek merkezdeki pikselin beş katından, onun alt, üst, sağ ve sol dört komşusunu çıkarır (köşelerin 0 olduğu buradan çıkar, ara adım eklendi):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Hoca sayfada canlı olarak bir katsayıyı -1'den -2'ye çevirerek sonucun biraz değiştiğini gösterdi. Yine de katsayıları tutarlı, doğru filtrelerden almak gerektiğini ekledi.

3.2 Köşegen Kenar Bulma

Slayttaki köpek fotoğrafında kenarlara bakarsak, örneğin kulağın çevresinde köşegen kenarlar (diagonal edge) vardır. Bunlar yatay ya da dikey olmayan, eğik çizgilerdir. Köşegenleri tespit etmek için bir köşegenin iki köşesine 1, öbür köşegenin iki köşesine -1 verilir (tersi de

düşünülebilir). Çekirdek bir köşegendeki değerleri alıp yardımcı köşegendeki değerleri çıkardığı için köşegen çizgiler belirginleşir.

3.3 Ortalama ve Bulanıklaştırma

En yaygın filtrelerden biri ortalama filtresidir. Hoca kısa bir not düştü: Bu aslında bir tür ortalamadır, ama doğrudan (düz) bir ortalama yapılmaz, katsayılarla ağırlıklandırılır. Her piksel konumuna göre önce 1 ile, sonra 2 ile, merkezdeki 4 ile çarpılır. Katsayıların toplamı 16 olduğu için sonuç 16'ya bölünür:

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Katsayıların yerleşimi hocanın sonradan Gauss filtresi için verdiği tariften çıkarıldı (ara adım eklendi): köşelerden 1 ile başlar, ortaya doğru 2 olur, en ortada en yüksek değer gelir. Gauss yoğunluk fonksiyonu çan eğrisi şeklinde olduğu için katsayılar da bu biçimi izler. $4+8+4=16$ toplamı da söylenen değeri tutar.

Hoca canlı gösterimde box blur ve Gauss blur filtrelerini de uyguladı. Boyut 5×5 yapıldığında daha fazla katsayı işin içine girer. Bu katsayılar gelişigüzel değildir, bir hesabı vardır: 3×3'te 1, 2, 4 iken 5×5'te 1, 4, 6 gibi değerler görülür.

> **Ders sonrası not:** Altyazıda Gauss filtresi için bir yerde "Gauss gürültüsü" geçiyor. Söz konusu olan gürültü değil bulanıklaştırma filtresidir (Gauss blur). Hoca box blur için de "ağırlıklı aritmetik ortalama" dedi; box blur'da katsayılar büyük olasılıkla eşittir, yani ağırlıksız düz ortalamadır.

3.4 Nihai Hedef: Röntgen Görüntülerinden Hastalık Tahmini

Hoca bu uygulamaların nihai hedefini bir örnekle anlattı. Elinizde göğüs hastalıklarıyla ilgili bir veri seti olsun: zatürre, COVID ve normal röntgen görüntüleri. Bu görüntülerden yola çıkarak kişide hangi hastalık olduğu tahmin edilebilir. Bunun için önce konvolüsyon işlemleri yapılır, görüntü üzerinde işlem yapa yapa ilerlenir, en sonda ise hastalıklıysa 1, değilse 0 tahmin edilecek şekilde bir etiketleme yapılır. Yapı bir evrişimli yapay sinir ağıdır.

Canlı gösterimde bir röntgen görüntüsü de açıldı. Keskinleştirme uygulandığında bazı kısımlar daha çok ortaya çıkıyor, bu da akciğer doktorunun yorumuna ek bilgi sunabiliyor. Görüntünün negatifini veren filtreler de vardır.

3.5 Birim Çekirdek

Soru: Ortada 1, çevrede 0 olan birim (identity) çekirdeği bütün görüntüde gezdirirsek ne elde ederiz? **Cevap:** Aynı resmi. Çünkü pikselin tam ortasındaki değer alınır, çevredekiler sıfırla çarpılır ve alınan değer ortaya yazılır. Tek fark, kenarlardan birer piksel kaybıdır: 500×500 görüntü 498×498 olur, çünkü $500-3+1=498$. Yukarıdan, aşağıdan, sağdan ve soldan birer piksel gider.

3.6 Farklı Amaçlara Farklı Filtreler

Filtre bankaları vardır; bu bankalardan amaca uygun filtreler seçilir. Her filtre her amaçla kullanılmaz. Örneğin bazı görüntülerde siyah ve beyaz noktalar görürsünüz: bir yerde siyah nokta, bir yerde beyaz nokta. Buna tuz-biber gürültüsü (salt and pepper noise) denir ve hoca sayfada bu gürültülü bir örnek gösterdi. Bunu yok etmek için farklı filtreler kullanılır. Keskinleştirme ise görüntü iyileştirmede işe yarar: yüksek çözünürlüklü bir görüntüde `sharpen` uygulanınca yaprağın üzerindeki belli noktalar daha keskin hâle gelir.

Yapılan şey özünde şudur: bir matris içinde başka bir matrisi dolaştırmak.

4. Örüntü Eşleme (Pattern Matching)

Hoca ardından string konusuna kaldığı yerden devam etti. Daha önce string fonksiyonları hızlıca konuşulmuştu, şimdi bir örüntü eşleme örneği incelendi.

Kullanıcıdan iki string okunur. Program, ikinci stringin birincinin hangi indisinde başladığını söyler. Word'de Ctrl+F ile metin aramak gibi düşünebilirsiniz: aradığınız metin orijinal metnin kaçıncı karakterinden itibaren geçiyor? Metne `text`, aranan şeye `pattern` denir.

4.1 Parametreler Neden İşaretçi Olabiliyor?

Ana programda `#define` ile tanımlanmış sabit uzunluklu iki dizi (`str1`, `str2`) var, fonksiyon ise bunları `char *` olarak alıyor.

Soru: Burada elemanlar dizi şeklinde tanımlıyken parametre neden işaretçi olabiliyor? Dizi olarak almamız gerekmiyor mu? **Cevap:** Hayır, çünkü dizinin adı (etiketi) aslında bir işaretçidir, ilk elemanın adresidir.

Sınıftan gelen bir cevap, işaretçiyi değer değişikliklerini dışarı taşımak için kullandığımız yönündeydi. Hoca bunun buradaki gerekçe olmadığını söyledi. Bu fonksiyon zaten orijinal stringleri değiştirmez, yalnız aranan stringin başlangıç indisini ya da -1 gönderir. Değişiklik yapacak olsaydı, parametreyi `char d[10]` gibi dizi olarak almak da yeterdi; içeride yapılan değişiklik yine orijinal diziye yansırdı. Yani bunun işaretçi kullanıp kullanmamayla ilgisi yoktur. Burada işaretçi, örneği farklı biçimde göstermek için özellikle seçilmiştir.

4.2 Kayan Pencere

Önce iki stringin uzunluğu gerekir, bunu `string.h`'taki hazır `strlen` verir. Aranan örüntü 3 karakterli olsun. İlk arama metnin ilk üç karakteri üzerinden yapılır. Bulunamazsa bir sonraki üç karakter, sonra onun yanındaki üç karakter denenir. Bu, az önceki konvolüsyona benzer şekilde bir kayan pencere (sliding window) gibi ilerler.

Son bakılacak yer, örüntünün metnin sonuna tam oturduğu konumdur. Başlangıç indisi daha ileri giderse, örüntü metnin dışına taşar. Bu yüzden `i` sıfırdan başlar ve en fazla uzun stringin uzunluğundan kısa stringin uzunluğu çıkarılarak bulunan değere kadar gider: `i <= len1 - len2`.

4.3 `indexOfV1`

Her konumda karakterler tek tek karşılaştırılır. İlk karakterler eşitse ikincilere bakılır. Eşit değilse sonrakilere bakmadan bir sonraki konuma geçilir. Kaç karakterin eşleştiğini `matchCount` tutar. Döngü iki şart sağlandıkça sürer: gösterilen karakterler eşit olacak ve `matchCount`, ikinci stringin uzunluğuna ulaşmamış olacak. Her adımda `matchCount` ve iki işaretçi birer artırılır.

Bu döngüden iki sebeple çıkılabilir:

- **Karakterler eşit değildir.** Örneğin `ABCD` içinde `ABL` arıyorsunuz; A'lar eşleşti, B'ler eşleşti, L ile C eşleşmedi. Aranan örüntü burada değildir ama ilerleyen konumlarda olabilir.
- **`matchCount` ikinci stringin uzunluğuna ulaşmıştır.** Kontrol edilebilecek en fazla karaktere erişilmiştir, yani aranan bulunmuştur. O konumun indisi olan `i` başlangıç noktasıdır ve doğrudan `return i;` yapılır.

Bulunamadıysa işaretçiler bir sonraki denemeye hazırlanır. `ptr1`, `matchCount - 1` kadar geri alınır, `ptr2` ise `matchCount` kadar geri alınarak örüntünün başına döner. `matchCount` de sıfırlanır. Bütün konumlar biterse `return -1;` ile çıkılır.

```
int indexOfV1(char *ptr1, char *ptr2)
{
    int len1 = strlen(ptr1);
    int len2 = strlen(ptr2);
    int i, matchCount = 0;

    for (i = 0; i <= len1 - len2; i++) {
        while (*ptr1 == *ptr2 && matchCount != len2) {
```

```
            matchCount++;
            ptr1++;
            ptr2++;
        }
        if (matchCount == len2)
            return i;

        ptr1 -= (matchCount - 1);
        ptr2 -= matchCount;
        matchCount = 0;
    }
    return -1;
}
```

Soru: Bir eşleşme bozulunca metinde neye dönmeliyiz? Hoca bunu `C, A, A, B` karakterlerinin eşleşip `C` ile `L`'nin eşleşmediği bir örnekte sordu. **Cevap:** Denemeye başlanan karakterin bir sonrakine, yani ilk A'ya. Şu ana kadar eşleşen karakter sayısı dörttür, işaretçi eşleşmeyen konumdadır. Üç karakter geri alınınca denemenin başlangıcından bir sonraki konuma gelinir.

> **Ders sonrası not:** Hoca geri alma miktarı için bir yerde "match etmeyen karakter sayısı kadar" dedi. Kodda geri alma, eşleşen karakter sayısının bir eksiği kadardır. Hiç karakter eşleşmemişse `matchCount - 1 = -1` olur, `ptr1` bir ileri gider ve yine bir sonraki konuma gelinmiş olur. `ABCABAC` metni ve `ABL` örüntüsü için konum konum izleme python ile hesaplandı, kod da `gcc -Wall` ile çalıştırıldı: eşleşme sayıları sırayla 2, 0, 0, 2, 0'dır ve sonuç -1'dir.

4.4 `indexOfV2`: `strstr`

İkinci versiyon `string.h` içindeki hazır `strstr` fonksiyonunu kullanır. Bir stringin içinde başka bir stringi arar. Bulursa bulduğu yerin adresini döndürür, `NULL` dönerse aranan yoktur ve fonksiyon -1 gönderir.

```
int indexOfV2(char *ptr1, char *ptr2)
{
    char *p = strstr(ptr1, ptr2);

    if (p != NULL)
        return (int)(p - ptr1);
    return -1;
}
```

> **Ders sonrası not:** İki fonksiyon aynı `main` içinde `gcc -Wall` ile derlenip denendi. `ABCD/ABC` için ikisi de 0, `ABCD/ABL` ve `ABCABAC/ABL` için ikisi de -1 verdi. `strlen` ve `strstr` için `<string.h>` eklenmelidir.

4.5 Çalışma Sorusu: Versiyon 3

Hoca mevcut kodun verimsiz bir yanına dikkat çekti. Yukarıdaki algoritmada C, A, A, B eşleşip sonraki karakter eşleşmeyince bir sonraki konuma dönülür, oradan yeniden denenir. Aynı karakterlere tekrar tekrar bakılır, gereksiz yere döngü sürer.

Örnek olarak `A, B, C, A, B, A, C` metninde `A, B, L` arandı. A'lar eşleşir, B'ler eşleşir, L eşleşmez. Bir sonraki konuma dönmek yerine, eşleşmeyen karakterin sonrasına gidilmesi önerildi: eşleşme tam orada bozulduğu için önceki karakterlerden başlayan denemelerin bu şartı sağlamayacağı söylendi. Bunu farklı bir algoritmayla versiyon 3 olarak kodlamak çalışma sorusu olarak verildi.

> **Ders sonrası not:** Hoca eşleşmeyen karakterin sonrasına gitmeyi "çok daha doğru" olarak önerdi; bu atlama bazı durumlarda gerçek eşleşmeyi kaçırır. Kısa notta verilen örnek: metin `AAAB`, örüntü `AAB`. İndis 0'dan `AA` eşleşir, üçüncü karakterde `A` ile `B` eşleşmez. Eşleşmeyen konumun ötesine atlayınca indis 1'deki eşleşme kaçar. Python ile hesaplandı: atlamalı yöntem -1, `indexOfV1` ve `strstr` ise 1 verdi (`indexOfV1` ve `strstr` için `gcc -Wall` ile de çalıştırıldı). Kısa notta adı geçen Knuth-Morris-Pratt

(KMP) algoritması, örüntünün kendi içindeki tekrarları hesaba katarak ilerler; python ile yazılıp denendi, aynı örnekte 1 verdi. Versiyon 3'ü kodlarken yönteminizi bu örnekle sınayın.

5. Matris İçinde Örüntü Arama

Hoca yeni bir kod yazarak matrislerin fonksiyona aktarımını gösterdi. Senaryo şudur: Stringlerimiz matris formatında olsun, her satırı bir kelime gibi düşünelim ve her satırda arama yapalım. Bazı parçalar yazıldı, bazıları kopyala-yapıştırla geldi.

5.1 Tanımlar

En büyük boyutlar `#define` ile verilir: `#define MAXROW 10`, `#define MAXCOLUMN 10`. Bir `char text[MAXROW][MAXCOLUMN]` metni ve aranan için bir `pattern` dizisi tanımlanır; aranan şeye genellikle `pattern` denir. `pattern`'in boyutu da `MAXCOLUMN`'dur, çünkü olabilecek en büyük değer budur. `M` ve `N` değişkenlerine kullanıcının satır ve sütun değerleri okunur. Matris 10×10 tanımlıdır, ama belki 5×5'lik kısmı kullanılacaktır. Fonksiyonlara bu yüzden `M` ve `N` de gönderilir.

5.2 `fillMatrix`

Soru: `fillMatrix(text, M, N);` çağrısında dönen bir değer var mı? **Cevap:** Hayır, bir değişkene atama yapılmıyor. O zaman fonksiyonun dönüş tipi `void` olur.

Matrisi rastgele karakterlerle doldurmak için `rand` fonksiyonu kullanılır. `rand`, `int` sınırları içinde rastgele bir değer döndürür. Bunun 3'e göre modu alınırsa ancak 0, 1 veya 2 çıkabilir. Üzerine `'A'` eklenir; hatırlanacağı gibi `'A'`'nın değeri 65'tir. Sonuç aralığı 65, 66 veya 67 olur, 68 olmaz. `char`'a dönüştürülünce A, B veya C elde edilir. İşlem önceliği yüzünden tüm ifade parantez içine alınır, böylece dönüşüm toplamın sonucuna uygulanır:

$$\texttt{rand() \% 3} \in \{0,1,2\} \quad \Rightarrow \quad \texttt{rand() \% 3 + 'A'} \in \{65,66,67\}$$

Değer aynı döngüde `%2c` ile ekrana da yazılır, iç döngü sonunda satır atlanır, böylece matris biçiminde görünür.

5.3 Sütun Boyutu Neden Zorunlu?

Matris parametre olarak alınırken olabilecek en yüksek boyutlar yazılabilir. Satır sayısı yazılmayabilir, çünkü dizi adı tek başına zaten bir işaretçidir. Fakat bu işaretçinin her satırda ne kadar uzağa gideceğini bilmek gerekir, bu yüzden sütun sayısı mutlaka yazılır. Hoca sütun boyutunu silip derledi ve derleyicinin orada mutlaka bir değer istediğini gösterdi. Burada yazılan değer en büyük sütun boyutudur, gerçek çalışma boyutları `M` ve `N` ile verilir.

> **Hocanın vurgusu:** Matrisi fonksiyona `[][MAXCOLUMN]` biçiminde gönderirken sütun boyutunu mutlaka yazın.

5.4 Örüntüyü Okumak

Kullanıcıdan örüntü `scanf("%s", pattern)` ile okunur.

Soru: Burada neden `&` yazmadık? **Cevap:** Çünkü `pattern` zaten bir dizidir, yani kendisi bir adrestir.

> **Ders sonrası not:** Hoca okuma sırasında bir boşluk bırakmanın sorun önlediğini hatırlattı. `gcc -Wall` ile denendi: `%d`'den sonra boşluksuz `%s` ile okuma doğru sonuç verdi, yani `%s` için bu boşluk gerekmez. Hocanın hatırlattığı sorun büyük olasılıkla `%c` ile okuma içindir.
>
> **Ek bilgi (derste anlatılmadı):** `scanf("%s")` boşlukta durur ve dizi boyutunu denetlemez, uzun girişte dizi taşabilir. `char pattern[10]` için `scanf("%9s", pattern)` en fazla 9 karakter okur (`gcc -Wall` ile denendi).

5.5 Örüntünün Uzunluğu

`search` fonksiyonunda önce örüntünün uzunluğu hesaplanır. Parametre `char pattern[]` ya da `char *pattern` olarak alınabilir, ikisi de aynı noktaya gelir. Hoca önce şunu yazdı:

```
while (pattern[len++] != '\0');
```

`ABC` için bu kod 4 verdi. Çünkü son karşılaştırmada `'\0'` bulunduğunda bile `len` artırılmıştır. Artırma döngünün içine alınırsa uzunluk 3 çıkar: en son karşılaştırma yanlış olduğunda gövde çalışmaz ve `len` artmaz. Diğer çözüm, ilk biçimi kullanıp sonra `len`'i bir azaltmaktır.

```
while (pattern[len] != '\0')
    len++;
```

> **Ders sonrası not:** `ABC` için iki biçim `gcc -Wall` ile çalıştırıldı, sırayla 4 ve 3 verdi; derste ekranda görülen sonuçla aynıdır.

5.6 `search`

Problem satır bazlıdır, sütun bazlı arama istenmiyor. `M` satır olduğuna göre `r` değişkeniyle satırlar gezilir. `r` asıl olarak, bulunan yerin kaçıncı satırda olduğunu söylemek için kullanılır ("üçüncü satırın beşinci sütununda" gibi).

Her satırda, biraz önceki gibi, başlangıç indisi için bir sınır gerekir: `bound = N - len`. Hoca değişkeni kodun ortasında tanımlamaktan vazgeçip başa aldı.

> **Hocanın vurgusu:** Yapısal programlama yapıyoruz, değişkenleri her yerde kafamıza göre tanımlamayalım; blok başında tanımlayalım.

İç döngüde `i`, 0'dan `bound`'a kadar gider ve aramanın başladığı sütunu tutar; `i` sabit tutularak örüntü boyunca `j` ile ilerlenir. `j` her başlangıçta sıfırlanır. `while` döngüsü iki şartla sürer: `j < len` ve `text[r][i + j] == pattern[j]`.

Hoca bir önceki kodla farkı vurguladı: Önceki kodda iki işaretçi birlikte artıyordu, burada yalnız `j` artıyor. Tek değişken hem metinde hem örüntüde ilerlemeyi sağlıyor. Bu döngüden iki sebeple çıkılır: Karakterler eşit değildir ya da `j == len` olmuştur. İkincisinde aranan bulunmuştur. Bulunan sayısını `count` tutar. `count` en sonda hâlâ sıfırsa hiçbir şey bulunamamıştır ve mesaj verilir.

Adımlar şöyledir:

1. `len` hesaplanır, `bound = N - len` bulunur, `count = 0` yapılır.
2. Her `r` satırı için `i = 0`'dan `bound`'a kadar her başlangıç denenir.
3. Her denemede `j = 0` ile başlanır; `j < len` ve karakterler eşit olduğu sürece `j` artırılır.
4. `j == len` ise `count` artırılır, `r` ve `i` yazdırılır.
5. Bütün satırlar bitince `count == 0` ise "not found" yazılır.

```
#include <stdio.h>
#include <stdlib.h>
#define MAXROW 10
#define MAXCOLUMN 10

void fillMatrix(char text[][MAXCOLUMN], int M, int N)
{
    int i, j;

    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++) {
            text[i][j] = (char)(rand() % 3 + 'A');
            printf("%2c", text[i][j]);
        }
        printf("\n");
    }
}
```

```c
void search(char text[][MAXCOLUMN], int M, int N, char pattern[])
{
    int r, i, j, len = 0, bound, count = 0;

    while (pattern[len] != '\0')
        len++;
    bound = N - len;

    for (r = 0; r < M; r++)
        for (i = 0; i <= bound; i++) {
            j = 0;
            while (j < len && text[r][i + j] == pattern[j])
                j++;
            if (j == len) {
                count++;
                printf("Pattern starts at %d. row and %d. column\n", r, i);
            }
        }

    if (count == 0)
        printf("not found\n");
}

int main(void)
{
    char text[MAXROW][MAXCOLUMN];
    char pattern[MAXCOLUMN];
    int M, N;

    printf("Enter row value: ");
    scanf("%d", &M);
    printf("Enter column value: ");
    scanf("%d", &N);

    fillMatrix(text, M, N);

    printf("Enter pattern: ");
    scanf("%s", pattern);

    search(text, M, N, pattern);
    return 0;
}
```

5.7 Derste Yapılan Çalıştırmalar

Hoca önce 5×5 bir matris üretip `BC` aradı. Program 0. satır 3. sütun, 1. satır 3. sütun ve 3. satır 1. sütunda eşleşme buldu, hoca bunları ekrandaki matrisle karşılaştırıp doğruladı. Sonra 7×7 bir matriste `CAA` aradı. Program üç konum buldu, ilki ikinci satır üçüncü sütundu, hoca bunları da ekranda gösterdi.

> **Ders sonrası not:** Derste ekrandaki matrisler altyazıda yer almadığı için bu konumlar yeniden üretilemedi. Kod `gcc -Wall` ile uyarısız derlendi. Bu makinede 5×5'te `BC` ve 7×7'de `CAA` aramaları, ekrana basılan matrisle elle karşılaştırılarak doğru konumları verdi; `rand` farklı ortamlarda farklı matris ürettiği için konumlar dersteki ile aynı değildir. Örüntü sütun sayısından uzunsa `bound` negatif olur, döngü hiç çalışmaz ve "not found" yazılır (bu da denendi). Bu kodda kullanıcı `M` veya `N` için en büyük boyuttan büyük değer girerse dizi taşar, derste bu kontrol yazılmadı.
>
> **Ders sonrası not:** Hocanın "değişkenleri her yerde tanımlayamayız" uyarısı bir üslup kuralı olarak yukarıda aktarıldı. `gcc -Wall` ile denendi: kodun ortasında tanımlama hata vermeden derlendi, yalnız `-std=c89 -pedantic` ile uyarı alındı. Yani bu,

güncel derleyicide dilin yasağı değildir.

Ek bilgi (derste anlatılmadı): `srand` çağrılmadan `rand` her çalıştırmada aynı sayı dizisini üretir; iki ardışık çalıştırmada aynı karakterler geldi (`gcc -Wall` ile denendi). Her çalıştırmada farklı matris için `main` başında `srand(time(NULL));` çağrılır ve `<time.h>` eklenir.

Hoca buradaki amacı açıkça belirtti: Hedef bir string algoritması öğretmek değil, oluşturulan matrisin fonksiyona nasıl aktarılacağını göstermektir. İleride işaretçiyle de aktarılacaktır. Dizi sembolüyle aktarılacaksa sütun bilgisi verilmelidir, çalışma sırasında ise gerçek boyutlar kullanılır.

6. Çok Boyutlu Diziler ve İşaretçiler

6.1 İndis ve İşaretçi Gösterimi

Çok boyutlu dizilere indis üzerinden erişirken yazılan `arr[1][2]`, arka planda şuna dönüşür:

$$\texttt{arr[1][2]} \equiv \texttt{*(*(arr + 1) + 2)}$$

Hoca 3×3 bir matris çizdi. `arr[0]` ilk satırın, `arr[1]` ikinci satırın başlangıcını gösterir. Dizi iki boyutlu olduğu için `arr[0]` ve `arr[1]` birer adrestir. `arr[1]`'e 2 birim eklenirse o satırın başından iki hücre ileri gidilir. Önüne yıldız konmazsa elde edilen adrestir, yıldız konunca oradaki değer gelir. Dizilerden hatırlanacağı gibi `arr[1]`, `*(arr + 1)` demektir. Sonuçta:

1. `arr + 1`: ikinci satıra gidilir.
2. `*(arr + 1)`: o satırın başlangıç adresi alınır.
3. `*(arr + 1) + 2`: iki hücre ileri gidilir.
4. `*(*(arr + 1) + 2)`: o adresteki değer alınır.

Aynı mantık üç boyutta bir yıldız daha ekler: `arr + 3`, `+ 4`, `+ 5` diye hesap edilir, `*(*(*(arr + 3) + 4) + 5)`. Hoca bunu ileride daha ayrıntılı konuşacaklarını söyledi.

Hoca toparlarken `arr[3][3]` üzerinden şunu söyledi: `arr` matrisin başlangıç adresini verir. Belli bir satırın adresine ulaşmak bir işaretçi işlemi, o satırda başka bir hücreye ulaşmak ikinci bir işaretçi işlemidir. Bu yüzden `arr + 1` için "işaretçinin işaretçisi" dedi.

Ders sonrası not: Hoca `arr + 1` için "işaretçinin işaretçisi" dedi; `int arr[3][3]` için `arr + 1`'in tipi `int (*)[3]`'tür, yani 3 elemanlı `int` dizisini gösteren işaretçi. `int **` değildir. `gcc -Wall` ile denendi: `int **` parametreli fonksiyona `arr` gönderilince derleyici tip uyuşmazlığı bildirdi ("expected `int **` but argument is of type `int (*)[3]`"). Aynı testte `sizeof(*(arr + 1))` 12 bayt (bir satır), `sizeof(**(arr + 1))` 4 bayt verdi. İki yıldızla erişim yine de doğrudur, `arr[1][2]` ile `*(*(arr + 1) + 2)` aynı değeri verdi.

6.2 İlk Değer Verme

Çok boyutlu dizide ilk değer verme biçimine göre sonuç değişir. `int exap[5][3]` için asıl biçim şudur:

```
int exap[5][3] = {{1, 2, 3}, {4}, {5, 6, 7}};
```

Her iç süslü parantez bir satırdır. Birinci satırda üç eleman da verilmiştir. İkinci satırda yalnız ilk eleman verilmiştir, diğerleri otomatik 0 olur. Üçüncü satır 5, 6, 7'dir. Kalan satırlar otomatik olarak 0 olur:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 0 \\ 5 & 6 & 7 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Değerler iç parantez olmadan tek dizi gibi verilirse:

```c
int exap[5][3] = {1, 2, 3, 4, 5, 6, 7};
```

Sütun sayısı 3 olduğu için elemanlar üçer üçer satırlara yerleştirilir: 1, 2, 3 birinci satıra, 4, 5, 6 ikinci satıra, 7 üçüncü satırın başına gelir, gerisi 0 olur. Biraz önce fonksiyona verilen sütun bilgisiyle aynı mantıktır:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

> **Hocanın vurgusu:** Bu iki ilk değer verme durumu önemlidir, farkına dikkat edin.
>
> **Ders sonrası not:** İki tanım `gcc -Wall` ile derlenip ekrana yazdırıldı, yukarıdaki matrisler çıktı. İkinci biçim geçerlidir, ama `gcc -Wall` bunun için `missing braces around initializer` uyarısı verir.

6.3 İşaretçi Dizisi ve İşaretçinin İşaretçisi

Önceki derslerde tek bir işaretçi üzerinden konuşulmuştu, örneğin `char *p;`. İstenirse bir işaretçi dizisi de oluşturulabilir: `char *p[5];` beş işaretçiden oluşan bir dizidir. Bunun stringlerde kullanımı vardır, daha sonra bakılacak; şimdilik yalnız tanımlamaya bakmak yeterli.

İşaretçinin işaretçisi de derste konuşulmuştu: Bir `int` değerin adresini `int *` tutar, o `int *` değişkeninin adresini tutan ise `int **`'dır. Bu genelleştirilebilir, 3 yıldızlı, 4 yıldızlı işaretçiler yazılabilir.

Dersin Toparlaması

Bu derste bir önceki haftalarda görülen dizi, döngü, işaretçi ve fonksiyon bilgileri iki uygulamada birleştirildi: Pisagor üçlüsünde iç içe döngülerle bütün kombinasyonlar tekrarsız denendi, konvolüsyonda dört iç içe döngüyle bir matris başka bir matrisin üzerinde gezdirildi, `do-while` ile girdi denetlendi. Konvolüsyonun görüntü işlemede ve evrişimli sinir ağlarında ayırt edici özellik çıkarmak için neden kullanıldığı filtre örnekleriyle gösterildi.

İkinci kısımda aynı kayan pencere fikri stringlerde ve matris satırlarında örüntü aramaya uygulandı. Matrisi fonksiyona verirken sütun boyutunun neden zorunlu olduğu, `arr[i][j]` ile `*(*(arr + i) + j)` denkliği, ilk değer verme biçimleri, işaretçi dizileri ve işaretçinin işaretçisi görüldü. Hoca sonraki derste işaretçi ve matris konusunu kısaca tamamlayıp dinamik bellek yönetimine (dynamic memory allocation) geçeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.